LitGraph documentation

Everything this tool does, how it does it, and — just as importantly — what it **can't** do. Written twice: once in plain English, once for engineers. Use the **Both / Plain English / Technical** switch above to pick your voice.

Version **2.0** · released 2026-07-15 · created by **Shreyan Kundu** · download this as a PDF. The PDF always contains both voices regardless of the switch.

What it is

IN PLAIN ENGLISH

Give LitGraph an arXiv paper — paste its id, type its title, or upload the PDF — and it draws you a map. The map shows the paper in the middle, the older papers it was built on, papers on similar topics, and everyone who wrote them. You can click any dot to read about it, and click again to grow the map outward.

TECHNICAL

A vanilla-JS Cytoscape.js (https://js.cytoscape.org) frontend over a pure-standard-library Python HTTP server, sourcing open scholarly data from the Semantic Scholar Graph API (https://api.semanticscholar.org). Runtime dependencies are `requests` and `pypdf` — no web framework, no ORM, no build step. Nodes are papers and authors; edges are `cites`, `similar` and `authored`.

Three kinds of thing end up on the map:

On the map	What it means	Where it comes from
Blue dot — referenced paper	A paper your seed paper cites. Older work it built on.	`/paper/{id}/references`
Purple dot — similar paper	Related work. Semantic Scholar decides what counts as similar, not us. See which corpus it draws from.	`/recommendations/v1/papers/forpaper/{id}`
Green square — author	A person. Linked to every paper of theirs on the map.	Rides along free inside each paper record
Gold dot — your seed paper	The paper you asked about. Always kept, even without an arXiv id.	Resolved via `arXiv:{id}` or title search

WHY AUTHORS MATTER

Authors are the glue. The same person is a single dot no matter how many papers they wrote, so two authors who worked together meet at the paper they share, and someone who shows up on lots of papers becomes an obvious hub. That's the "how is this field connected" view.

What's new in v2.0 this version

v2.0 is about **control and honesty**. v1.0 gave you three fixed depth presets and no say in anything. Now you choose what to fetch, how much, and what counts as important — and the app tells you the truth about the parts that don't work well.

- **You set the limits** — a limit box per source, capped at what the API genuinely allows.
- **Priority** — rank by most cited, newest, or oldest.
- **A References toggle** — v1.0 had toggles for everything *except* references.
- **Your limit finally means what it says** — see the fetch pipeline.
- **Citing papers removed** gone — and here's the honest reason.
- **This page**, plus a PDF of it.

> **TECHNICAL**
>
> The `MODES` presets became data in the same shape as a user config; `normalize()` validates and clamps any incoming config against the API's real ceilings. `/api/analyze` and `/api/expand` now accept an optional `config` object, and `/api/config` publishes `max_limits`, `sorts` and `presets` so the UI derives its own constraints from the server rather than hardcoding them.

How to use it

1. Type an arXiv id, a title, or upload a PDF. Any of these shapes work: `1706.03762`, `1706.03762v7`, `arXiv:1706.03762`, or a pasted `arxiv.org/abs/…` link.
2. Optionally set how many of each thing you want, and what to prioritise.
3. Hit **Analyze**.
4. Click any dot for its abstract and arXiv link.
5. Click **Expand** in the side panel to grow the map from that dot.

> **TWO USEFUL BEHAVIOURS**
>
> + **Add** queues several papers into one shared map — papers they both cite show up once, in the middle, which is a quick way to see how two lines of work connect. And anything you've looked at before loads instantly, because it's cached.

The controls

One row above the graph, four groups. Set them and hit Analyze.

Group	What it does
References	**Limit** — how many of the papers your seed cites to show (max 1000). **Priority** — which ones, when there are more than your limit.
Similar	**Limit** — how many related papers (max 500). **Priority** — Most relevant keeps Semantic Scholar's own ranking.
Authors	**Per paper** — how many authors to draw per paper (max 50). **From** — every paper, or only the seed's.
2nd hop	Take your most-cited references and pull *their* references too.

The **Quick / Deep / Complex** dropdown is just a shortcut: it loads a set of starting numbers into these controls. Nothing stops you changing them afterwards.

> **TECHNICAL**
>
> The controls serialise straight into the `config` object documented under HTTP API, which `normalize()` validates and clamps server-side. Any source set to `0` returns before an HTTP call is made, so zeroing a limit costs nothing — that's how you isolate a single source if you want to study one on its own.

> **NOTE**
>
> **2nd hop** takes your most-cited references and pulls *their* references too — 15 each. It multiplies the work, so it's off by default except in the Complex preset, which sets it to 6.

Limits & the real caps

> **IN PLAIN ENGLISH**
>
> Every tab has a box for how many papers you want, and next to it the biggest number that's actually allowed. Type something bigger and it snaps back. These aren't numbers we invented — they're the real ceilings Semantic Scholar enforces.

Setting	Cap	How we know
References per paper	**1000**	1001 returns `400 'limit' must be <= 1000`
Similar papers	**500**	501 returns `400 Limit must be between 0 and 500`
Papers per author	**1000**	1001 returns `400 'limit' must be <= 1000`
Authors per paper	**50**	Our own limit — author lists come embedded, so this is just readability
2nd hop refs to expand	**25**	Our own limit — each one costs another request

TECHNICAL

Ceilings live in `sources.MAX_LIMIT` and were established empirically by probing the live API for the 400 it returns one past the edge. They're enforced in three places: the number input's `max` attribute, a JS clamp on change, and `graph_builder._clamp()` server-side. The server clamp is the one that matters — a hand-rolled `curl` asking for 99999 gets a clamped **200**, never a 400 from upstream. `/api/config` publishes `max_limits` so the UI never hardcodes them.

NOTE

Paging is capped at `offset + limit < 10000`, so no more than 10,000 rows of any one list are reachable even in principle. It doesn't bite for references (papers rarely cite more than a few hundred things).

Priority (sorting)

IN PLAIN ENGLISH

If a paper has 200 references and you only want 40, *which* 40 matters. Priority decides: the 40 most-cited, the 40 newest, or the 40 oldest. "Most cited" is usually what you want — it surfaces the classics.

- **Most cited** — the influential ones first.
- **Newest first** — what's happening lately.
- **Oldest first** — the roots of the idea.
- **API order** — whatever Semantic Scholar hands back, unsorted.

TECHNICAL

`sources.sort_papers()`. Missing values are handled so ranking never throws: `citationCount` falls back to `0`, `year` to `0` descending / `9999` ascending, which parks undated papers at the bottom either way. Sorting happens **after** the arXiv filter and **before** the trim — see the pipeline — and we do it ourselves for the reason in the next section.

The toggles

IN PLAIN ENGLISH

The three checkboxes on the graph — References, Similar papers, Authors — just show and hide things. They never re-fetch, so they're instant, and nothing is lost: untick and tick, and it's all back.

TECHNICAL

`applyFilters()` keys off **edge relationships**, not node types. It has to: a paper node's `type` can't express that a paper is both a reference and a similar paper — first write wins — so the relationship only truly exists on the edge. The function hides edges by `rel`, then drops any paper with no surviving structural edge (`cites` / `similar`), then re-checks, iterating to a fixed point (authors can be stranded by a paper that was itself stranded). Author edges deliberately don't count as structural, or a reference would cling to the graph via its authors after you unticked References. Visibility is computed in plain JS first and applied inside a single `cy.batch()`, so no style is ever read back mid-batch.

EXPORT BEHAVIOUR

Hidden nodes are excluded from **PNG** export (they aren't rendered) but are still present in **JSON** export, which always contains the full graph.

Expanding & batching

IN PLAIN ENGLISH

Click a paper and hit **Expand** and its own references and similar papers join the map. Click an author and expand, and you get the rest of their work. The map grows instead of starting over, and the button locks so you can't expand the same dot twice.

TECHNICAL

`POST /api/expand` with `{id, type}`. The builder starts from an empty node/edge map and has no knowledge of what's already on screen, so it returns its whole neighbourhood and `mergeIntoGraph()` discards ids the client already holds. The clicked node is never returned — only referenced by id in the new edges. Layout re-runs with `randomize:false` so existing nodes keep their positions. Expand uses the fixed `EXPAND` config (25 refs, 8 similar, 30 author papers), **not** the control-bar settings. The expanded flag lives in browser state only and is forgotten on re-analyze.

Batching: `+ Add` queues several seeds into one build. Because nodes are keyed by Semantic Scholar's `paperId` and authors by `author:{authorId}`, anything two seeds share is deduplicated into a single node automatically — which is what visually connects them.

Coverage: the arXiv filter

IMPORTANT

LitGraph keeps **arXiv papers only**. Everything else — books, journal-only papers, older work, anything paywalled — is silently dropped. This is usually what's shrinking your graph, **not** your limit.

WHAT THIS MEANS IN PRACTICE

Coverage varies sharply by field and by era. For a representative machine-learning paper, only a small minority of its references — frequently around one in ten — are present on arXiv. The rest are real and correctly cited; they simply have no open-access arXiv record, so they cannot carry a guaranteed free full-text link. Raising your limit will not recover them.

This is a deliberate trade, not a bug. It's what lets every dot on the map open a free full text. The cost is that the map is a view of *the open-access slice* of the literature, not the whole of it.

TECHNICAL

`arxiv_id_of()` reads `externalIds.ArXiv`; papers without one are filtered out in `_fetch()`. The seed itself is exempt and always kept. Observed survival rates through the filter run at roughly 10–35% depending on the paper and the source. Relaxing the filter to include non-arXiv papers — rendered distinctly and without a PDF link — is a configuration change rather than a rewrite, and would increase reference coverage by roughly an order of magnitude.

Where "similar" comes from

IN PLAIN ENGLISH

Semantic Scholar's recommendation service can search two different pools: only **recent** papers, or the **whole corpus**. It defaults to recent — which for a literature survey is unsuitable, because "recent" restricts results to newly posted preprints that have not yet accumulated citations. LitGraph asks for the whole corpus.

WHY THE DEFAULT IS UNSUITABLE

Corrected in v2.0. The service's default pool restricts recommendations to recently posted work. For an established paper this returns either nothing at all, or a set of new preprints that have not yet accumulated citations — in both cases, no useful neighbourhood, and no error to indicate the problem.

Requesting the full corpus returns the recognised related work for the same paper: the same endpoint, the same single request, one parameter different.

TECHNICAL

`sources.similar()` sends `from=all-cs`. The parameter is undocumented in the obvious places and defaults to `recent`, so it's easy to never discover — nothing errors, you just quietly get bad recommendations. We fall back to the default pool when all-cs returns nothing, because all-cs is the **computer science** corpus and arXiv is not only CS: a physics or biology paper is better served by the default than by an empty graph.

NOTE

This endpoint rate-limits harder than the rest of the API and doesn't always honour an API key. If similar papers come back empty for a paper that should have neighbours, wait a moment and retry.

Result ordering

IN PLAIN ENGLISH

Semantic Scholar's API looks like you can ask it to sort results. You can't — it accepts the request, says "OK", and ignores it. So LitGraph downloads a big batch and does the sorting itself.

VERIFICATION

Passing `sort=citationCount:desc` to `/paper/{id}/citations` returns **200** with rows in the same order as no sort at all. The clincher: `sort=bogusfield:desc` — a field that cannot exist — also returns **200** with identical rows. An API that validated its sort key would reject that. So the parameter is accepted and discarded, and passing it would be a silent no-op that *looks* like a working feature. All ordering is therefore done in `sort_papers()` after fetching.

A useful consequence. Because we fetch a pool and sort locally, changing a limit doesn't need the network at all — the pool is already cached, so we just re-slice it. Asking for 10 references instead of 40 is instant.

Removed in v2.0: citation lookup removed

IN PLAIN ENGLISH

v1.0 could show you papers that *cite* your paper — its descendants. It's a genuinely useful idea, and it was removed because it couldn't be made to work honestly.

The problem is scale plus ordering:

- A seminal paper may be cited on the order of **100,000** times. v1.0's Deep mode displayed **15** of them.
- The API returns them in a recency-biased order and — per above — cannot sort. So those 15 were 2026 papers with **0 citations each**: the most obscure work that ever touched it.
- Fetching a 1000-paper pool and ranking it ourselves didn't rescue it. The most-cited paper in that pool had **33 citations**. BERT, GPT and the rest — the actual descendants worth seeing — weren't in the pool at all.
- Paging can only reach 10,000 of ~100,000, and the same bias applies throughout.

Design rationale. The feature could have shipped with a limit box and a priority dropdown. It would have appeared identical to the other sources and behaved as though it worked. But sorting a biased sample just gives you a tidily-ordered biased sample — a feature that misrepresents its own fidelity is worse than no feature. It was cut rather than presented as functional.

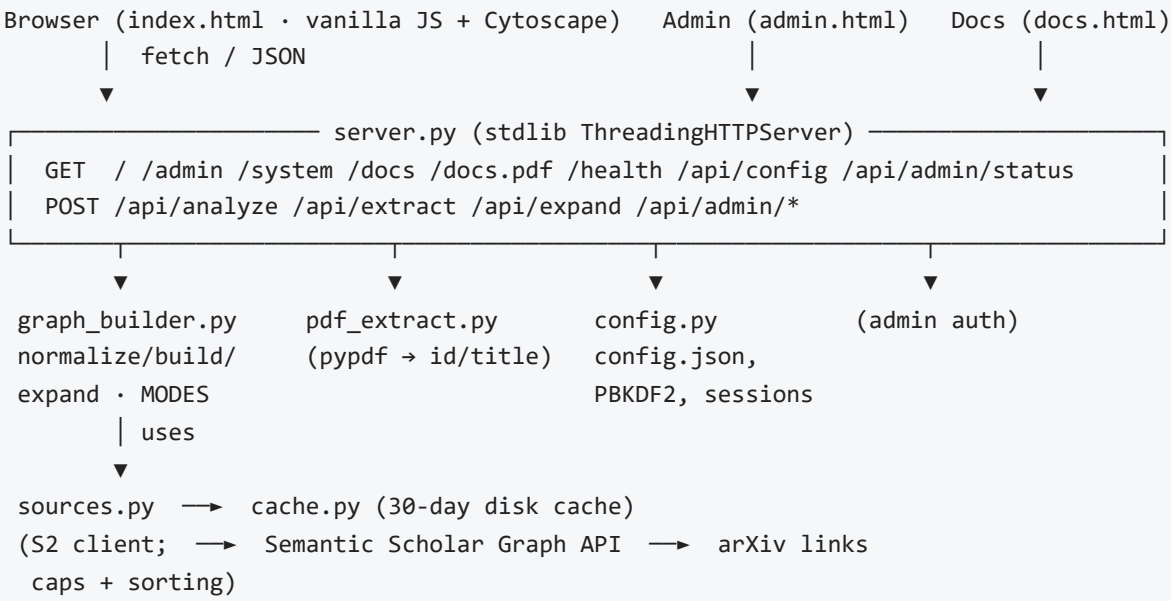
> **TECHNICAL**
>
> Removed outright, not disabled: `SemanticScholar.citations()`, the `cited_by` edge type and its styling, its controls, its toggle, and the `cites` key from every preset and from `EXPAND`. A stale client sending a stale config is ignored rather than erroring. Expand is one API call cheaper as a result. Restoring it would mean re-adding one client method and one edge type — the pipeline would take it back unchanged.

Known limitations

Stated explicitly, so that none of it is a surprise in deployment:

- **Non-arXiv papers are invisible.** See above — this is the big one.
- **No "who cited this".** Removed in v2.0, reasons here.
- **Similar papers can still come back empty** for very new or non-CS papers, and it fails quietly — no purple dots, no message. Much rarer since v2.0 (why), but not impossible.
- **Reference lists are Semantic Scholar's, not the PDF's.** Upload a PDF and we only read its arXiv id or title out of it; the bibliography comes from S2, and can be incomplete for very new papers.
- **PDF title detection is a heuristic.** It takes the biggest text on page 1, which is right for normal papers but can misfire on unusual layouts (a huge cover heading, a slide deck). An arXiv id inside the PDF is always the reliable path — and when one is present the title isn't used to identify the paper at all.
- **Old-style arXiv ids aren't auto-detected.** Both the PDF scanner and the id parser match the post-2007 scheme (`1706.03762`) only, so `hep-th/9901001` isn't recognised inside a PDF. You can still paste one into the box — it's passed to Semantic Scholar untouched.
- **The frontend needs internet** even for cached data — Cytoscape loads from a CDN.
- **Author identity is only as good as S2's.** Same-name researchers may merge or split; we key on `authorId` and inherit whatever it says.
- **Sessions live in memory.** Restart the server and you're logged out.
- **Titles beat ids for accuracy.** A title search takes the first hit, which can be the wrong paper. An arXiv id is exact — prefer it.

Architecture



File	Job
`backend/server.py`	Routing, cookie sessions, config validation entry point. No framework.
`backend/graph_builder.py`	The core. `normalize()` validates configs; `_fetch()` is the pipeline; `build()` merges seeds; `expand()` grows.
`backend/sources.py`	S2 client. Owns `MAX_LIMIT`, `sort_papers()`, throttling, backoff, graceful 429s.
`backend/cache.py`	Disk JSON cache, 30-day TTL, best-effort (never fails a request).
`backend/config.py`	`VERSION`, config.json, PBKDF2 auth, sessions, stats.
`backend/pdf_extract.py`	pypdf → arXiv id or title.
`frontend/index.html`	The app: tabs, controls, graph, filters, expand, export.
`frontend/docs.html`	This page.

The fetch pipeline

> **IN PLAIN ENGLISH**
>
> The order these four steps happen in is the difference between "give me 40 references" meaning 40, and it meaning 4.

```
POOL  ──►  arXiv FILTER  ──►  SORT  ──►  TRIM to your limit
 ▲                                        ▲
always the API's max                      your limit applies HERE, last
```

PRIOR BEHAVIOUR

v1.0 asked the API for exactly your limit and *then* dropped the non-arXiv results: a request for 40 references returned 40 rows, most of which were then discarded by the arXiv filter, leaving a handful on screen. The limit was spent on records that were about to be thrown away. Filtering before trimming means a request for 40 now yields up to 40 *arXiv* references — every one the paper has.

WHY THE POOL IS ALWAYS THE MAXIMUM

Two reasons, both free wins:

- A big fetch costs the **same single HTTP call** as a small one — only bytes differ, and rate limits count calls. A typical complete reference list is well under 0.1 MB; a full 1000-row pool with abstracts is a few megabytes.
- A constant pool size means a **constant cache key**. Your limit is applied after the cache, so changing it re-slices cached data with **zero** API calls.

Implementation: `graph_builder.POOL` mirrors `sources.MAX_LIMIT`; `_fetch()` does pool → filter → `sort_papers()` → `[:limit]`. A limit of `0` short-circuits before any request.

HTTP API

Method · Path	Purpose
GET `/` `/admin` `/system` `/docs`	The four pages
GET `/docs.pdf`	This documentation as a PDF
GET `/health`	`{ok:true}`
GET `/api/config`	`{version, released, has_key, default_mode, is_setup, max_limits, sorts, presets}`
POST `/api/analyze?mode=`	Build a graph. JSON body, or a raw PDF with `Content-Type: application/pdf`
POST `/api/extract`	Raw PDF → `{arxiv_id, title}`
POST `/api/expand`	`{id, type}` → new nodes/edges
POST `/api/admin/*`	`setup`, `login`, `logout`, `apikey`, `settings`, `password`, `clear-cache`

The analyze body

```
POST /api/analyze
{
  "seeds": [ {"arxiv_id": "1301.3781"}, {"query": "attention is all you need"} ],
  "config": {
    "refs":    {"limit": 40, "sort": "citations"},    // max 1000
    "similar": {"limit": 15, "sort": "default"},      // max 500
    "authors": {"enabled": true, "scope": "all", "per_paper": 8},
    "second_hop": 0                                   // max 25
  }
}

// sort: "citations" | "year_desc" | "year_asc" | "default"
// Omit "config" entirely to use the ?mode= preset (quick | deep | complex).
```

VALIDATION CONTRACT

Every field is optional and every value is clamped, so the endpoint is safe to poke at. Over-max clamps down; negatives clamp to 0; non-numeric falls back to the preset's value; an unknown `sort` is ignored in favour of the default. Errors: **400** no usable seed, **422** nothing resolvable (or a total rate-limit failure), **200** with a `meta.warnings` array for partial results. The response echoes the fully-resolved config at `meta.config` — so you can always see exactly what ran.

Cache & rate limits

IN PLAIN ENGLISH

Every answer from Semantic Scholar is saved to disk for 30 days. Papers you've already looked at load instantly and don't touch the network. This is what makes the tool comfortable to use for free.

TECHNICAL

`cache.py`: key is a SHA-256 of URL + params, stored as JSON under `LITGRAPH_DATA_DIR/.cache/`, 30-day TTL, written atomically via temp + `os.replace`. Writes are best-effort and never fail a request. Throttling in `sources.py` is a process-wide lock: **0.4 s** between calls with an API key, **1.2 s** without. On 429/502/503/504 it retries up to 5 times with a 1.8× backoff from 3 s. Persistent 429 sets `rate_limited`, appends a warning and returns partial results — **422** only if nothing at all resolved. It never 500s on a rate limit.

No API key is needed. A free key from semanticscholar.org/product/api (https://www.semanticscholar.org/product/api) raises the limits and can be pasted into Admin, or set as `S2_API_KEY`.

Auth & admin

IN PLAIN ENGLISH

The first time you open **/admin** you set a password. After that it protects your API key, the default depth, and the cache controls.

TECHNICAL

PBKDF2-HMAC-SHA256, 200,000 iterations, 16-byte random salt, compared with `secrets.compare_digest`. Login mints a `secrets.token_urlsafe(24)` session in an **HttpOnly**, **SameSite=Lax** cookie with an 8-hour TTL. Sessions are in-memory and cleared on restart. The key and password hash live in `config.json` inside `LITGRAPH_DATA_DIR` (`backend/` by default), which is gitignored. `/api/config` exposes only a `has_key` boolean; the key itself is only ever returned masked, to an authenticated session.

Running it

```
# One-time setup
pip install -r requirements.txt   # requests, pypdf — that's the lot

# Run it
python backend/server.py          # -> http://localhost:8000
                                  # (or double-click run.bat on Windows)

# Rebuild this documentation PDF after editing docs.html
python tools/build_docs_pdf.py    # needs playwright (dev-only, not a runtime dep)
```

Stop it with **Ctrl+C**. Try `1301.3781`, `1706.03762` or `1810.04805`.

DATA LOCATION

`config.json` (admin password hash + API key) and the response cache sit in `backend/` by default, and both are gitignored. Set `LITGRAPH_DATA_DIR` to move them somewhere else — useful if you want the code on a different disk to the data, or a shared cache between checkouts. Delete `config.json` to reset first-run setup. Set a port with `PORT`.

ABOUT THIS PDF

`/docs.pdf` is a static artifact built from this page by headless Chromium, so the server keeps its two-dependency diet rather than shipping a PDF engine. It is **not** regenerated automatically — edit `docs.html` and re-run the script, or the PDF will drift out of date.

Data & licensing

IN PLAIN ENGLISH

LitGraph doesn't own any of the paper data — it borrows it from Semantic Scholar every time you run a search. That means two things: you need your own free API key, and the credit line on the graph ("Paper data from Semantic Scholar") has to stay where it is.

Resource	Link
API product page & key request	semanticscholar.org/product/api (https://www.semanticscholar.org/product/api)
Academic Graph API reference	api.semanticscholar.org/api-docs/graph (https://api.semanticscholar.org/api-docs/graph)
Recommendations API reference	api.semanticscholar.org/api-docs/recommendations (https://api.semanticscholar.org/api-docs/recommendations)
Tutorial	semanticscholar.org/product/api/tutorial (https://www.semanticscholar.org/product/api/tutorial)
API License Agreement	semanticscholar.org/product/api/license (https://www.semanticscholar.org/product/api/license)

ATTRIBUTION IS A LICENCE CONDITION

The API License Agreement requires an attribution to "Semantic Scholar" on the site using it. LitGraph shows one in the graph legend. **It must not be removed.**

TECHNICAL

Endpoints used, requested fields, the published rate limits LitGraph is engineered against, caching policy, third-party component licences, and the licence terms in full — including the **CC BY-NC** exposure that applies to commercial deployment — are documented in `DATA-AND-LICENSING.md` in the repository. Operators supply their own key; none is bundled.

LitGraph is not affiliated with, endorsed by, or sponsored by the Allen Institute for Artificial Intelligence, Semantic Scholar, or arXiv.

Changelog

v2.0 — 2026-07-15 current

- **Added** per-source limits, capped at the API's verified real ceilings.
- **Added** priority: most cited / newest / oldest / API order.
- **Added** a References toggle — v1.0 had one for everything else.
- **Added** this documentation page, plus its PDF, and a real version number.

- **Fixed** limits being spent before the arXiv filter ran — "40 references" could yield 4. Pipeline is now pool → filter → sort → trim.
- **Fixed** toggles stranding orphan dots, and being unable to hide references at all. Filtering is now edge-driven.
- **Fixed** pasted arXiv ids being rejected in every shape but one. `1706.03762v7` (the arXiv filename), `arXiv:1706.03762` (what the paper stamps on itself), a pasted `arxiv.org/abs/…` URL, or an id with a stray space all returned "could not identify this paper". Now all fold to the same id, and a "title" that's really an id is recognised as one.
- **Fixed** a 500 when a paper's reference list came back `null` — `.get("data", [])` returns `None` when the key exists and is null, so the default never applied and the loop crashed.
- **Fixed** a corrupt or empty upload returning a 500 stack trace. It's a 422 with a readable message now.
- **Fixed** JSON export leaking an internal `_expanded` UI flag into every exported node.
- **Fixed** similar papers returning no or irrelevant results — the recommendations service was left to default to its *recent-papers* pool, which returned **no results at all** for some well-known papers and uncited preprints for others. Now drawn from the whole corpus (detail).
- **Fixed** PDF title detection selecting the longest line rather than the largest, so a paper carrying a licence banner or notice above its title had the banner extracted as the title. Detection now uses font size, and ligatures are normalised.
- **Changed** limit changes now re-slice the cache instead of hitting the network.
- **Removed** citing papers — reasons.
- **Removed** Docker packaging (Dockerfile, compose file, .dockerignore). It's a two-dependency Python app; `pip install` and `python backend/server.py` is the whole story now.

v1.0

- Interactive Cytoscape graph; references, citing papers, similar papers, authors.
- Three fixed depth presets; PDF upload; batching; node expansion.
- Admin panel, disk cache, system map, guided tour, PNG/JSON export.

LitGraph v2.0 · Created by Shreyan Kundu · Data from Semantic Scholar (https://api.semanticscholar.org) · PDF · System map · App